Feasibility Report: SD-Card Flashing & STM32 Remote Programming

Goal: Make the spot welder self-contained for firmware updates (no PC):
1. Flash **ESP32** firmware from the on-board microSD slot.
2. Flash **STM32G474CE** firmware via the ESP32 (from WiFi upload or microSD).

Scope: Analysis only — nothing implemented. Findings are based on the current firmware, board definition, the project pin-assignment doc, and ST application notes (AN2606 / AN3155).

---

1. Current SD-Card Support

Item	Finding
SD library in `platformio.ini`	**None.** `lib_deps` only has `rzeldent/es-p32_smartdisplay`. No `SD`, `SD_MMC`, `SdFat`, or `FS` usage.
SD code in `main.cpp`	**None.** No `SD.begin()`, no SPI bus for the card, no file I/O.
Hardware slot present?	**YES.** `boards/esp32-8048S043C.json` defines `BOARD_HAS_TF` and the pins below.

SD-card pins (from board definition)

```
TF_CS       = GPIO10
TF_SPI_MOSI = GPIO11
TF_SPI_SCLK = GPIO12
TF_SPI_MISO = GPIO13
```

- The microSD is on a **dedicated SPI bus** (4 pins, GPIO10–13).
- **No conflict** with the display (RGB parallel), touch (I2C GPIO19/20), or the STM32 UART (GPIO17/18). These four pins are otherwise unused in the firmware.
- **Conclusion: the SD card is fully accessible.** It just needs the standard Arduino `SD` / `SPI` libraries initialised on a dedicated `SPIClass` instance.

---

2. ESP32 ↔ STM32 Hardware Connections

UART link (confirmed, already in use)

Signal	ESP32	STM32	Notes
ESP RX ← STM32 TX	GPIO17	**PA9** (USART1_TX)	
ESP TX → STM32 RX	GPIO18	**PA10** (USART1_RX)	
Application config	—	—	2,000,000 baud, **8N1** (both sides)

Source: `main.cpp` (`STM32_TO_ESP32_PIN 17` , `ESP32_TO_STM32_PIN 18` , `STM32Serial.begin(2000000, SERIAL_8N1, ...)`) and `docs/PIN_ASSIGNMENTS.md` (PA9/PA10 = USART1 to ESP32).

BOOT0 control pin

- **NOT wired to the ESP32.** BOOT0 does not appear anywhere in the ESP32 code, and the STM32 pin table (`docs/PIN_ASSIGNMENTS.md`) does not route BOOT0 to the inter-board connector. On the WeAct STM32G474 module BOOT0 is on its own button/pad, not under ESP32 control.

RESET (NRST) control pin

- **NOT wired to the ESP32.** NRST is listed as “Hardware reset / Input / Infrastructure” (WeAct reset button + RC), but it is **not** connected to an ESP32 GPIO.

Is there any existing reset/boot/jump mechanism?

- ESP32 side: **none** (no GPIO toggling of STM32 boot/reset).
- STM32 side: **no** jump-to-system-bootloader routine in `main.c` (the only `0x1FFF…` reference is a factory-calibration read, unrelated).

---

3. STM32G474CE Bootloader Protocol (AN2606 / AN3155)

The STM32G474CE has a **built-in ROM system bootloader** — no code to write on the STM32 to receive an update; we only need a host that speaks the protocol.

Bootloader UART (AN2606, “STM32G47xxx/48xxx” section, p.238)

- **USART1 on PA9 (TX) / PA10 (RX)** is a supported bootloader peripheral.
- **This is the exact link the ESP32 already has.**
- Bootloader UART format: **8 data bits, EVEN parity, 1 stop bit (8E1)** with auto-baud (host sends `0x7F` so the bootloader locks onto the baud rate).
- ⚠️ The running app uses **8N1**; for bootloader mode the ESP32 must reopen the port as **8E1** (e.g. 115200 8E1). One-line `STM32Serial.begin()` change.

Protocol commands needed (AN3155)

Step	Bytes	Purpose
Sync / auto-baud	`0x7F` → expect `ACK 0x79`	Establish connection
Get	`0x00 0xFF`	Bootloader version + supported commands
Get-ID	`0x02 0xFD`	Confirm chip = G474
Erase	Extended Erase `0x44 0xBB` (mass or page list)	Clear flash
Write Memory	`0x31 0xCE` + addr+chk + 1–256 byte chunks	Program (start `0x08000000`)
Read Memory (optional)	`0x11 0xEE`	Read-back verify
Go	`0x21 0xDE` + addr	Jump to new app
Framing	every cmd = byte + complement; each block checksummed; each step ACK/NACK	Reliability
Note	Read-protection (RDP) must be off, else erase/write are blocked	

There is no off-the-shelf “stm32flash for ESP32-Arduino” library, but the protocol is small and well-documented; a focused C++ class (~300–500 lines) is the normal approach (mirrors the open-source `stm32flash` tool / Arduino `STM32duino` examples).

---

4. Feasibility Summary

4A. ESP32 firmware from microSD — ✅ HIGHLY FEASIBLE

- Slot + dedicated SPI pins exist and are free.
- ESP-IDF/Arduino has first-class support: `Update.h` can flash an OTA image read from a file (`SD.open("/firmware.bin")` → `Update.writeStream()`), then `ESP.restart()`. This is the standard “SD-card OTA” pattern.
- Reuses the OTA partition scheme already in place (16 MB flash, default partitions). The OTA color-loop display blanking work already done applies here too (blank panel while writing flash).

4B. STM32 firmware via ESP32 — ⚠️ FEASIBLE WITH A CAVEAT

The data path (UART) and the target (ROM bootloader on PA9/PA10) are ready.
The open question is **how to put the STM32 into bootloader mode**:

- **Option 1 — Hardware BOOT0 + NRST control (robust / recommended).** Wire 2 ESP32 GPIOs → STM32 BOOT0 and NRST. ESP32 drives BOOT0 high, pulses NRST → STM32 enters system bootloader every time, **even if the app is bricked**. Requires a small **hardware modification** (2 wires + the ESP32 must expose 2 free GPIOs — see §5).
- **Option 2 — Software "jump to bootloader" (no hardware mod).** Add a command to the STM32 app (e.g. `ENTER_BOOTLOADER`) that de-inits peripherals and jumps to system memory `0x1FFF0000`. ESP32 sends the command, then talks AN3155. **No wiring change.** Caveat: only works while the STM32 app is healthy enough to receive the command — a failed/corrupt flash can leave **no recovery path** without BOOT0/NRST.
- **Best practice: do both** — Option 2 for convenience, Option 1 as the un-brickable fallback.

---

5. ESP32 Free-GPIO Reality Check (for Option 1)

Pins already consumed: RGB panel (8,3,46,9,1,5,6,7,15,16,4,45,48,47,21,14, 39,40,41,42), backlight (2), touch (19,20,38), SD (10,11,12,13), STM32 UART (17,18), button (0). The ESP32-S3 OPI PSRAM (`qio_opi`) also consumes GPIO35–37, and GPIO43/44 are the USB-serial debug UART.

➡️ **Free GPIOs are scarce.** Driving BOOT0 + NRST needs **2 spare pins brought out on the board's expansion header** (the Sunton 8048S043C breaks out a handful on its IO/speaker connectors). This must be verified against the physical connector pinout before committing to Option 1.

---

6. Libraries / Code Needed

Feature	Needed
SD access	`SD.h` + `SPI.h` (bundled with Arduino-ESP32) on a dedicated `SPIClass`, pins 10–13.
ESP32 self-flash from SD	`Update.h` (bundled). Read `.bin` → `Update.writeStream()` → restart.
STM32 flashing	New custom `Stm32Bootloader` class implementing AN3155 (sync, Get-ID, erase, write, go) over `STM32Serial` reconfigured to 8E1.
STM32 image source	Either a `.bin` on SD, or received over WiFi (HTTP upload / the existing TCP bridge) and buffered to SD/PSRAM.
STM32 app change (Option 2 only)	Small `jump_to_system_bootloader()` + a UART command handler.

---

7. Complexity & Risk

Item	Complexity	Risk
ESP32 SD flashing	**Low–Med**	Low. Dual OTA partitions mean a bad image rolls back; bricking is unlikely.
STM32 flashing — protocol	**Medium**	Medium. AN3155 is simple but needs careful checksums, 8E1 switch, timeouts, retries, verify-after-write.
STM32 flashing — boot entry (Opt 1)	**Low (code) + HW mod**	**Brick risk if no fallback.** With BOOT0/NRST you always recover.
STM32 flashing — boot entry (Opt 2)	**Low–Med**	**Higher brick risk** — interrupted flash with no BOOT0/NRST = dead board until SWD/PC rescue.
EMI during STM32 flash	—	Welder must be **disarmed/idle**; verify after write; never flash while charged.

8. Recommended Implementation Approach (phased)

1. **Phase 1 — ESP32 SD flashing (quick win, low risk).**
   Add `SD` init on GPIO10–13; “Flash from SD” item on the Setup tab; read `/firmware.bin` via `Update.writeStream()`; blank panel during write; restart. Ships value immediately, no STM32 risk.

2. **Phase 2 — STM32 flashing over the existing UART (Option 2, software jump).**
   - Add `Stm32Bootloader` (AN3155) on the ESP32; reopen UART at 8E1.
   - Add `ENTER_BOOTLOADER` command + `jump_to_system_bootloader()` to the STM32 app.
   - Source the STM32 `.bin` from SD first (simplest), then add WiFi upload.
   - Always **read-back verify** before `Go`.

3. **Phase 3 — Hardware BOOT0/NRST fallback (Option 1, un-brickable).**
   Confirm 2 free header GPIOs, add the 2 wires, implement hardware boot-entry as the recovery path. Strongly recommended before relying on field STM32 updates.

9. Key Confirmed Facts (evidence)

- SD slot present, pins free — `boards/esp32-8048S043C.json` (`BOARD_HAS_TF`, `TF_CS=10/MOSI=11/SCLK=12/MISO=13`).
- No SD code today — `platformio.ini`, `src/main.cpp` (no SD/FS refs).
- STM32 UART = PA9/PA10 ↔ ESP32 GPIO17/18 @ 2 Mbaud 8N1 — `main.cpp`, `docs/PIN_ASSIGNMENTS.md`, `STM32G474CE/src/main.c` (`MX_USART1_UART_Init`).
- STM32G47x bootloader on USART1 PA9/PA10 @ 8E1 — AN2606 §47.1 (p.238).
- BOOT0 / NRST **not** wired to ESP32; no existing jump-to-bootloader — code search of both projects.